Efficient Thinking III: Efficient Judging

When an LLM judge is worth its compute — and the two conditions it must clear

Louay Alsakka · July 17, 2026 · *draft v0.1*

Abstract

After a model samples N candidate answers, something must select one. Three selectors exist: a verifier, exact but available only in checkable domains; majority vote, free wherever answers can be counted; and an LLM judge, available everywhere and costing compute. Papers I–II of this series measured what search extracts and what evaluators bound; this paper prices the judge. We ran a 48-cell grid — six judge sizes (1.5B–72B) against four policy sizes (0.5B–7B), at N = 4 and N = 16 candidates over frozen GSM8K caches, 150 problems per cell — in two protocols (pick-best-from-list and pairwise tournament), with every claimed effect tested by exact paired McNemar.

Judges earned wins in exactly one corner. Eight cells survive significance, all of them a strong judge (14B–72B) selecting for a weak policy (0.5B–1.5B), and six of the eight at N = 4. Every apparent win for a competent policy (3B–7B) is statistically null ($p = 0.22$–1.0). The pattern follows a two-factor law inherited from Paper II's proposition: a judge beats the vote only if its selection accuracy among covered problems exceeds the policy's mode-correctness — a condition that held with zero counterexamples across all 48 cells — and only where coverage admits a better answer to select, which gates the 16 cells where a competent judge still lost. Judge value = selection × coverage, the same decomposition, measured a third way.

Two further results price the judge honestly. First, allocation: every significant judge win is FLOP-dominated by a judge-free configuration — a bigger policy with free majority vote reaches higher accuracy at a fraction of the cost — so on a task where voting works, the optimal share of compute spent on judging is approximately zero, and our registered predictions of an interior optimum are scored as misses. Second, the N tax: no judge win survives at N = 16 in any protocol. Pick-best drowns by position — the 72B judge chose the first or last candidate in 52 of 60 trials against a uniform expectation of 15, blind to the middle of its own list — and pairwise tournaments compound elimination errors round by round. Below the competence floor a judge is worse than free: the 1.5B and 3B judges win nowhere and are significantly destructive in 8 of 16 pairwise cells. One sentence: pay for a judge only when the world gives you neither a vote nor a verifier — and then only a judge that clears the consensus it replaces, over a field small enough to actually read. All experiments ran on two Apple-Silicon machines over Paper II's frozen caches; claims are scoped to this benchmark and these budgets.

Results at a glance

finding	measurement	where
judges win in one corner only	8/48 cells significant, all weak-policy × strong-judge; every competent-policy "win" null	§3
the two-factor law	selection condition necessary in 48/48 (zero counterexamples); coverage gates the 16 predicted-only cells	§3
judge compute is never the right marginal spend here	each significant win FLOP-dominated by a judge-free config at higher accuracy	§4
the N tax is universal	six of eight wins at N = 4; nothing significant at N = 16 in either protocol	§5
the mechanism, photographed	52/60 choices at list edges (uniform ≈ 15); correct answers missed uniformly in the middle	§5
below the floor, worse than free	weak judges: zero wins, 8 significantly negative cells, worst -20.0	§5

1. Introduction

Paper II ended at a gap: over identical samples, a perfect verifier kept climbing while majority vote saturated, and an LLM judge captured part of the difference. That result priced nothing — it showed a judge *can* select better than a vote without saying when the judge's own compute is worth spending. This paper asks the priced question. At a fixed budget, a system designer chooses how to split compute between the policy that generates candidates and the judge that selects among them; call the judge's share s. The proposal for this paper registered six predictions about that split before any cell ran, including an interior optimum (P1) and a regime where a smaller policy with a larger judge beats the reverse (P5). The grid falsified both, and the falsifications are the paper's most useful content: on a benchmark where majority vote works, $s \approx 0$ is optimal everywhere we measured, and the conditions under which a judge earns any compute at all turn out to be exactly two, both measurable in advance.

Contributions, in the order the paper develops them: the 48-cell judge-policy grid with per-cell exact significance, locating all judge value in the weak-policy × strong-judge corner (§3); the two-factor law — selection above consensus as a necessary condition with zero counterexamples, coverage as the gate on sufficiency — connecting judging to Paper II's mode-versus-coverage proposition cell by cell (§3); the FLOP-dominance result scoring the registered allocation predictions as misses (§4); the N tax with its positional mechanism photographed, and the protocol taxonomy that follows (§5); and the scored-prediction ledger, which this time includes two analysis errors caught during the paper's own review (§7).

A note on statistical power, stated once. Every cell is n = 150 problems on GSM8K over Paper II's frozen 32-sample caches (sample once, select many); every claimed win or loss carries an exact paired McNemar p-value, and cells that do not reach $p < 0.05$ are called null regardless of their point estimate. This discipline deletes ten of the eighteen apparent judge wins. Single-benchmark scope is the paper's main limitation and §8 registers the direction of the expected difference on harder benchmarks.

2. The grid

Six judges (Qwen2.5-Instruct 1.5B, 3B, 7B, 14B, 32B, 72B, 4-bit) select among N ∈ {4, 16} candidates drawn from four policies (0.5B, 1.5B, 3B, 7B) on 150 GSM8K problems, against three references computed from the same samples: majority vote (free), oracle best-of-N (coverage), and the policy's greedy answer. Two protocols: **pick-best**, one call with all N candidates in context; **pairwise**, a single-elimination tournament of two-candidate calls. Token counts are logged per cell (mean generation tokens per candidate; mean judge input tokens), so every configuration carries its compute cost in params × tokens units. The full grid at both N, pick-best mode (**bold** = significant win over majority):

judge \ policy	0.5B	1.5B	3B	7B
1.5B judge 24.7 / 25.3	53.3 / 60.0	68.7 / 75.3	88.0 / 90.7	
3B judge 24.0 / 25.3	56.7 / 60.0	72.0 / 75.3	87.3 / 90.7	
7B judge 31.3 / 25.3	62.7 / 60.0	71.3 / 75.3	89.3 / 90.7	
14B judge **33.3** / 25.3	**68.0** / 60.0	76.0 / 75.3	88.7 / 90.7	
32B judge **36.7** / 25.3	**66.7** / 60.0	74.0 / 75.3	92.0 / 90.7	
72B judge **36.7** / 25.3	**71.3** / 60.0	75.3 / 75.3	93.3 / 90.7	
(oracle @N=4) 46.7	*77.3*	*86.7*	*96.0*	

judge \ policy	0.5B	1.5B	3B	7B
1.5B judge 25.3 / 36.7	56.0 / 64.0	58.7 / 86.0	86.7 / 92.7	
3B judge 24.0 / 36.7	56.7 / 64.0	58.0 / 86.0	89.3 / 92.7	
7B judge 28.7 / 36.7	66.0 / 64.0	67.3 / 86.0	85.3 / 92.7	
14B judge 35.3 / 36.7	69.3 / 64.0	72.7 / 86.0	90.0 / 92.7	
32B judge 42.7 / 36.7	**74.0** / 64.0	76.0 / 86.0	94.0 / 92.7	
72B judge 44.0 / 36.7	**71.3** / 64.0	70.7 / 86.0	86.7 / 92.7	
(oracle @N=16) 69.3	*92.0*	*94.7*	*96.7*	

3. Where judges win, and the law that says so — Anchor 1

Applying the no-win-without-significance rule to the grid leaves exactly eight cells:

judge	policy	N	Δ (pick − majority)	exact p
14B	0.5B	4	+8.0	0.0042
14B	1.5B	4	+8.0	0.0118
32B	0.5B	4	+11.4	0.0005
32B	1.5B	4	+6.7	0.0129
32B	1.5B	16	+10.0	0.0041
72B	0.5B	4	+11.4	0.0009
72B	1.5B	4	+11.3	0.0002
72B	1.5B	16	+7.3	0.0433

All eight are a strong judge selecting for a weak policy, and six of eight are at N = 4. The ten remaining positive cells — including every cell where a judge appears to beat a competent 3B or 7B policy — are statistically null (p = 0.07–1.0), and this paper calls none of them wins.

The corner is not an accident of scale but of arithmetic. Define q_judge as the judge's selection accuracy restricted to problems where a correct answer is present among the candidates, and mode-correctness as the policy's majority-vote accuracy — the strength of the free consensus the judge must beat. Across all 48 cells, **the condition q_judge > mode-correctness is necessary without exception**: 0 cells beat majority while failing it. It is not sufficient: 16 cells clear it and lose anyway, and each of those is gated by coverage — the judge selects well among the answers present, but too few problems have a correct answer present for good selection to overcome a strong vote. Judge value is therefore the product of two measurable factors, selection margin and coverage, which is Paper II's proposition — majority converges to the mode, oracles to coverage — operating inside a third experiment. The 2×2 over the grid: predicted-and-won 18, predicted-and-lost (coverage-gated) 16, unpredicted-win 0, neither 14.

The law's practical form: both factors are computable *before* deploying a judge — mode-correctness from a small sample of votes, q_judge from a small labeled calibration set — so whether a judge will pay is a measurement, not a bet.

4. The price: allocation, scored — Anchor 2

The proposal's central registered prediction (P1) was an interior optimum: some split s between policy and judge compute beating both endpoints. The grid says otherwise. Pricing each significant judge win against judge-free configurations from the same caches (cost in params × tokens per problem):

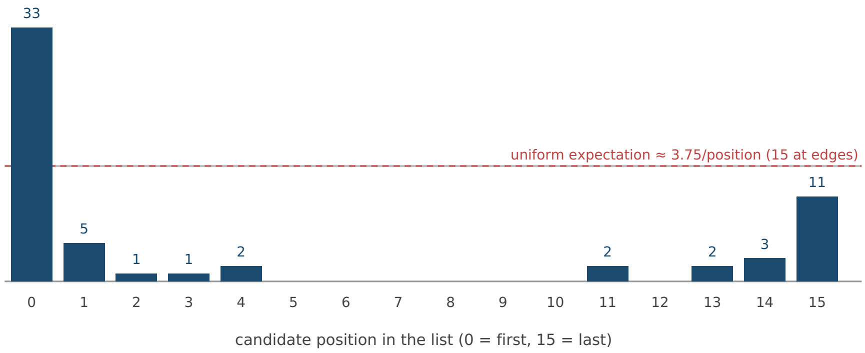
judge win (cell)	acc	cost	judge-free config	acc	cost	win costs
14B×0.5B N=4	33.3	29k	1.5B+majority@4	60.0	2k	17.9×
14B×1.5B N=4	68.0	19k	3B+majority@4	75.3	4k	4.9×
32B×0.5B N=4	36.7	64k	1.5B+majority@4	60.0	2k	40.1×
32B×1.5B N=4	66.7	41k	3B+majority@4	75.3	4k	10.6×
32B×1.5B N=16	74.0	151k	3B+majority@4	75.3	4k	39.0×
72B×0.5B N=4	36.7	144k	1.5B+majority@4	60.0	2k	89.4×
72B×1.5B N=4	71.3	90k	3B+majority@4	75.3	4k	23.2×
72B×1.5B N=16	71.3	332k	3B+majority@4	75.3	4k	85.7×

Every significant judge win is dominated — a judge-free configuration reaches strictly higher accuracy at a fraction of the cost, because policy FLOPs buy coverage and consensus simultaneously while judge FLOPs buy only selection over a fixed, weak candidate pool. P1 (interior optimum) and P5 (smaller policy + larger judge beats the reverse) are scored as misses; P2 (judge share grows with budget) inherits the miss, since s ≈ 0 at every budget we can construct from these cells. The exception that defines the judge's real niche is a deployment lock: when the policy is fixed — by memory, latency, or product constraints — and its consensus is weak, a strong judge is the best selector available, and the eight-win corner is exactly that regime. Paper II found the same structure for search itself: the lever pays on the constrained frontier, not the free one.

5. The N tax: two protocols, two failure modes, one mechanism

No judge win survives at N = 16 in either protocol, and the two protocols fail differently. In pick-best, accuracy *falls* as the candidate list grows — the weak-judge collapse ran to −14 points, and even strong judges lose their significant N = 4 wins by N = 16. The mechanism is positional. Instrumenting the 72B judge's choices at N = 16:

Figure 1 — where the 72B judge looks: chosen-candidate position, N = 16 (n = 60)



The judge chose an edge of the list 52 times in 60 (uniform expectation ≈ 15), overwhelmingly the first position, while the correct answers it missed were distributed uniformly through the middle — lost-in-the-middle [Liu et al. 2023], photographed in the act of deleting a judge's value. Sixteen candidates in context is effectively a choice among two.

Pairwise removes the list and pays a different tax. Re-running the two cells where the law predicted wins that pick-best failed to deliver (registered as R-a), with the N = 8 midpoint added:

judge	N	Δ pairwise − majority	exact p
7B	4	+8.7	0.0106
7B	8	-4.7	0.2962
7B	16	-1.4	0.8555
14B	4	+9.4	0.0066
14B	8	+6.0	0.0931
14B	16	+4.6	0.3105

Both judges win significantly at N = 4; nothing survives at N = 16 — the registered prediction that pairwise would flip the N = 16 cells to wins is scored a miss. Pairwise recovers most of the drowned deficit (+6 to +7 points over pick-best at N = 16) but compounds elimination error: each round is a fresh chance to discard the correct answer permanently, and the rounds grow with N. The residual margins order by the judge's selection gap over consensus, exactly as the two-factor law prescribes: the 14B judge (larger margin) ends positive-null, the 7B judge (marginal) at parity.

Below the competence floor, no protocol helps and the judge is actively harmful: across 16 pairwise cells, the 1.5B and 3B judges win nowhere, are significantly *worse* than free majority in 8 cells, and bottom out at -20.0 points (1.5B judge, 3B policy, N = 16). A weak judge does not merely waste its compute; it converts a working vote into a worse decision.

The protocol decision rule that falls out: keep candidate fields small (the value is at N = 4, in both protocols); use pick-best for its single-call cheapness when the judge clears the law's condition with margin; use pairwise only to rescue a strong judge from a list it would drown in; and never deploy a judge below the competence floor, where the correct protocol is no judge at all.

6. Synthesis, and the handoff

Assemble the pieces and the judge's contract is short. A judge is worth compute when — and only when — (1) no verifier exists, (2) the free vote is weak (low mode-correctness), (3) the judge's selection accuracy clears that vote (the law's necessary condition), (4) coverage admits something better to select, and (5) the candidate field is small enough to read. On GSM8K, a checkable task with strong consensus, conditions (1) and (2) already fail for every competent policy, and the marginal FLOP always belonged to the policy. That is this paper's negative result, and its value is the domain it points at: creative and aesthetic work, where there is no verifier, no meaningful vote over poems, and the judge is not an option but the only evaluator there is. Paper IV begins exactly there, carrying this paper's machinery — the pairwise protocol, small fields, the calibration-first measurement of q — into the regime where the judge is forced.

7. Registered predictions, scored

prediction (registered before the run)	outcome
P1 — an interior allocation optimum exists for mid-capability policies	**Miss.** s ≈ 0 everywhere measured; every significant judge win is FLOP-dominated by a judge-free configuration (§4)
P2 — optimal judge share grows with budget	**Miss, inherited.** With s ≈ 0 at all constructible budgets, no growth exists to observe (§4)
P3 — judge size beats judge search at equal FLOPs	**Unscored.** Judge-side self-consistency cells were not run; deferred to the ET-IV harness
P4 — q(j) rises smoothly with a competence knee	**Half hit: smooth yes, knee no.** Mean q_judge at N = 4 rises smoothly and monotonically — 1.5B 73%, 3B 75%, 7B 81%, 14B 85%, 32B 86%, 72B 89% — with no competence knee anywhere in q itself. The registered knee was real but misplaced: weak judges fail not because q collapses but because a respectable q still falls below a stronger consensus — the law's condition, not the judge's raw quality, is what has the threshold. At N = 16 the 72B point inverts, an artifact of position collapse (§5), not of judge quality
P5 — smaller policy + larger judge beats the reverse	**Miss.** Policy FLOPs dominate wherever the comparison can be built (§4)
P6 — pick-best beats pairwise at fixed judge FLOPs	**Split by regime.** Pick-best wins for mid judges (pairwise compounds their errors); pairwise wins for strong judges on weak policies at large N (it removes the drowning tax); neither survives N = 16 (§5)
Absolute competence knee at ~7B: judges beat majority from the same threshold as ET-II's crossover (in-conversation, pre-grid)	**Miss.** No judge size wins across policies; even 72B loses to a competent policy's consensus. Value is relative, not absolute (§3)
Relative competence: the 3B judge helps weak policies, loses to strong ones (in-conversation, pre-grid)	**Partial miss.** Direction right, floor wrong: the 3B judge helps nobody and is significantly harmful below the floor (§5)
N-degradation shrinks with judge size (in-conversation, pre-grid)	**Partial hit.** Monotone easing from 1.5B to 32B; the 72B reversal at N = 16 resolved by the position diagnostic as protocol, not capability (§5)
R-a — pairwise flips the two law-mismatch cells to wins at N = 16	**Miss as registered; mechanism confirmed.** Pairwise recovers the drowned deficit into parity, and both cells win significantly at N = 4; the N tax, not the list alone, is what binds (§5)
Negative control — weak judges win nowhere, even in pairwise	**Hit,** with a bonus severity: 8 significantly negative cells (§5)

Two analysis errors were caught during this paper's own review and belong in its record: an aggregate form of the two-factor law was briefly computed with a shared denominator, making it a tautology (24/24 agreement by identity), and a units mismatch then inverted it; both were caught before any conclusion rested on them, and the per-problem form reported in §3 is the corrected computation, independently reimplemented against the committed per-problem files. The program's rule — no delta believed until it survives a low-variance re-measurement — applied to its own instruments twice this cycle.

8. Limitations and future work

One benchmark: GSM8K's consensus is unusually strong (high mode-correctness), which is precisely the condition that starves judges; on harder benchmarks with weaker consensus the law predicts a larger judge-viable region, and MATH's per-policy mode-correctness

— computable from Paper II's existing caches — is the registered next measurement. One model family at one quantization, and the position-collapse result in particular should be checked against a bf16 judge before being read as a capability rather than a quantization interaction. n = 150 per cell bounds the detectable effect at roughly ±7 points; the corner wins clear it, the competent-policy nulls may hide real ±2-point effects we cannot see. Judge prompts were fixed and unoptimized; prompt engineering the judge is a confound we priced at zero and a reader should not. P3 remains unscored and moves to the ET-IV harness, where judge-side compute is the entire subject.

Reproducibility

All grids, per-problem outcome files, pairwise and diagnostic runs, and the analysis scripts (including the bridge computation and this draft's build script) are in the repository under `judging/`, generated over Paper II's frozen caches (`reasoning/`). Every number in this paper is computed from those files by script at build time; none is transcribed.

References

- Alsakka, L. (2026). *Efficient Thinking I–II.* This series.
- Brown, B. et al. (2024). *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling.* arXiv:2407.21787.
- Cobbe, K. et al. (2021). *Training Verifiers to Solve Math Word Problems.* arXiv:2110.14168.
- Liu, N. F. et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts.* TACL.
- Wang, X. et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models.* arXiv:2203.11171.
- Zheng, L. et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.* NeurIPS.